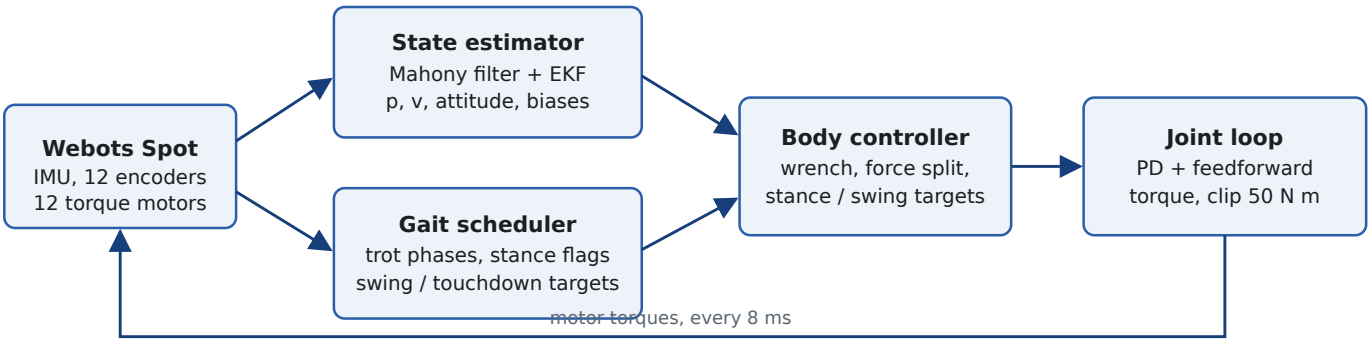
Spot Locomotion: Math & Control Architecture

A concise guide to every algorithm in the project: kinematics, state estimation, force control, trot gait, push resistance.
Boston Dynamics Spot in Webots, ROS 2 Jazzy, Python. Everything below is implemented and was run in simulation, except the MPC section at the end, which is the plan.

1. Big picture and frames
5. Force distribution and joint torques
2. Kinematics (IK, FK, Jacobian)
6. Trot gait and foot placement
3. State estimation (complementary filter, EKF)
7. How it resists pushes
4. Body controller (virtual model control)
8. Numbers, lessons, next step (MPC)

1. Big picture and frames



Everything runs **inside the Webots plugin, in the same 8 ms simulation step as the sensors**. Early on the controller was a separate ROS node; message delays of tens of milliseconds made the stiff body spring-damper unstable. Running in-step removes the latency and makes runs deterministic.

Frame / symbol	Meaning
World `W`, body `B`	ENU world (z up). Body x forward, y left, z up; origin at the Spot body origin.
p, v	Body position and velocity in the world. $R \in SO(3)$ rotates body vectors to world: $x_W = R\,x_B$.
ω	Body angular velocity (body frame). $[a]_\times$ is the skew matrix with $[a]_\times b = a \times b$.
r_i	Foot i position in the body frame; f_i its ground reaction force.
$g = 9.81$, $M = 27.2$ kg	Gravity and total robot mass (body 14.4 kg + 4 legs × 3.2 kg).

2. Kinematics

2.1 One leg: inverse kinematics (IK)

Each leg has 3 joints: hip abduction θ_1 (sideways swing), hip pitch θ_2, knee θ_3. Link lengths $l_1 = 0.358$ m (thigh), $l_2 = 0.392$ m (calf); the thigh is offset sideways from the abduction axis by $d_y = \pm 0.117$ m (left +, right −). Given a foot target (x, y, z) relative to the hip axis:

$$r = \sqrt{y^2 + z^2}, \qquad \theta_1 = \operatorname{atan2}(y, -z) - \arcsin\frac{d_y}{r}$$

$$z' = -\sqrt{r^2 - d_y^2}, \qquad d^2 = x^2 + z'^2, \qquad \cos\theta_3 = \frac{d^2 - l_1^2 - l_2^2}{2\,l_1 l_2}$$

$$\theta_2 = \operatorname{atan2}(x, -z') - \operatorname{atan2}\big(l_2 \sin\theta_3,\ l_1 + l_2\cos\theta_3\big)$$

> **INTUITION**
> First rotate the whole leg sideways so the foot lies in the leg's plane (θ_1). In that plane it is a two-link arm; the law of cosines gives the knee angle, and the hip pitch aims the thigh so the foot lands at the target. If $|\cos\theta_3| > 1$ the target is out of reach and the code raises an error instead of guessing.

2.2 Forward kinematics (FK) and Jacobian

$$x = l_1\sin\theta_2 + l_2\sin(\theta_2+\theta_3), \qquad z' = -\big(l_1\cos\theta_2 + l_2\cos(\theta_2+\theta_3)\big)$$
$$y = d_y\cos\theta_1 - z'\sin\theta_1, \qquad z = d_y\sin\theta_1 + z'\cos\theta_1$$

The foot velocity relative to the body follows from the **Jacobian** $J = \partial r/\partial q$ (computed by central finite differences): $\dot r = J\,\dot q$. It is used twice: for leg odometry (§3.3) and to turn foot forces into joint torques (§5).

2.3 Whole body and the Webots joint convention

For a desired body pose (position p, rotation R) and a foot target f^W in the world, the foot relative to the hip axis in the body frame is $R^\top(f^W - p) - h_i$ (hip mount h_i); feed it into the leg IK. This keeps the feet fixed on the ground while the body moves.

> **GOTCHA: WEBOTS ZERO IS NOT A STRAIGHT LEG**
> At joint angle 0 the thigh points 35.6° backward and the calf 39.9° forward. The IK angles are converted to motor angles by
>
> $$q_{\text{hip}_x} = -\theta_1, \qquad q_{\text{hip}_y} = \theta_2 + 0.622, \qquad q_{\text{knee}} = \theta_3 - 1.317 \quad (\text{rad})$$
>
> Limits: abduction $[-0.6, 0.5]$, hip pitch $[-1.7, 1.7]$, knee $[-0.45, 1.6]$; torque limit 50 N·m. These constants were read from the Webots PROTO files and verified by checking that FK roll/pitch agrees with the inertial unit to about 0.3°.

3. State estimation

The controller needs p, v, R. Nothing measures them directly. We have an IMU and joint encoders, each flawed in a different way:

Sensor	Model	Weakness
Gyro	$\omega_m = \omega + b_g + n_g$	Bias b_g: integrating gives attitude that drifts.
Accelerometer	$a_m = R^\top(a_W - g_W) + b_a + n_a$, $g_W = (0,0,-g)$	Reads +g when still. Integrating twice drifts fast. Also senses body acceleration, so gravity is only visible when the body isn't accelerating.
Encoders + kinematics	foot position r_i, velocity $\dot r_i = J\dot q$	No drift, but only informative about the world when a foot is planted.

The IMU is fast but drifts; the legs don't drift but only work when feet are on the ground. Fusion uses each where it is strong. Simulated bias and noise are added to the Webots IMU (it is perfect by default) so the filter has something to estimate; the true biases are known so we can check it.

3.1 Complementary filter: roll and pitch from gyro + accelerometer

Trust the gyro at high frequency, the accelerometer's gravity direction at low frequency (Mahony filter). Each sample:

$$u_{\text{pred}} = R^\top e_z, \qquad u_{\text{meas}} = \frac{a_m}{|a_m|}, \qquad e = u_{\text{meas}} \times u_{\text{pred}}$$
$$\omega_c = \omega_m - \hat b + k_p\,e, \qquad \dot{\hat b} = -k_i\,e, \qquad q \leftarrow q \otimes \exp(\omega_c\,\Delta t)$$

e is the rotation axis that tilts the predicted "up" toward the measured one. The correction is skipped when $\big||a_m| - g\big| > 0.8$ (the robot is accelerating, so the accelerometer is not a gravity reference). Gravity cannot

see rotation about the vertical, so **yaw only follows the gyro**. Result in Webots: roll/pitch error 0.1–0.6°. Its attitude is also used to decide contact (§3.4) because it is independent of the EKF.

3.2 Leg odometry: body velocity from a planted foot

A foot in stance does not move in the world: $f_i = p + R\,r_i = \text{const}$. Differentiate:

$$v + R\big(\dot r_i + \omega \times r_i\big) = 0 \quad \Longrightarrow \quad v_B = R^\top v = -\big(\dot r_i + \omega \times r_i\big)$$

$\dot r_i = J\dot q$ is the foot's velocity relative to the body (the body moves forward, the foot slides backward under it); $\omega \times r_i$ accounts for body rotation. Every planted foot gives one measurement of the body velocity; four feet average the noise.

3.3 Error-state Extended Kalman Filter (EKF)

> **WHY "EXTENDED" AND "ERROR-STATE"**
> A Kalman filter is the optimal estimator when you have a model (how the state evolves) and noisy measurements. Rotations make the problem nonlinear, so we linearise around the current estimate: the *extended* filter. A rotation lives on a curved space (a unit quaternion has 4 numbers for 3 degrees of freedom), so instead of filtering q directly we keep a *nominal* state propagated exactly, and filter a small *error* δx whose dynamics are very nearly linear. After each correction the error is folded into the nominal state and reset to zero.

Nominal state: position p, velocity v, attitude q, accelerometer bias b_a, gyro bias b_g. **Error state** (15 values): $\delta x = [\delta p, \delta v, \delta\theta, \delta b_a, \delta b_g]$ with $q_{\text{true}} = q \otimes \exp(\delta\theta)$.

Prediction (every IMU sample, 125 Hz)

$$a_W = R(a_m - b_a) + g_W, \quad p \leftarrow p + v\,\Delta t + \tfrac{1}{2}a_W\Delta t^2, \quad v \leftarrow v + a_W\Delta t, \quad q \leftarrow q \otimes \exp\big((\omega_m - b_g)\Delta t\big)$$

Error dynamics $\dot{\delta x} = F\,\delta x + \text{noise}$, biases modelled as random walks:

$$\delta\dot p = \delta v, \qquad \delta\dot v = -R[a]_\times\delta\theta - R\,\delta b_a, \qquad \delta\dot\theta = -[\omega]_\times\delta\theta - \delta b_g$$

$$F_d = I + F\Delta t + \tfrac{1}{2}F^2\Delta t^2, \qquad P \leftarrow F_d P F_d^\top + Q$$

Q holds the IMU noise (accelerometer noise grows velocity error, gyro noise grows attitude error) and the bias random walks. P is the filter's own uncertainty: it gets bigger every prediction and smaller every correction.

Corrections

Every correction has the same form, with innovation y (measured − predicted) and a Jacobian H mapping the error state to the measurement:

$$S = HPH^\top + R_m, \quad K = PH^\top S^{-1}, \quad \delta x = Ky, \quad P \leftarrow (I - KH)P(I - KH)^\top + KR_mK^\top$$

$$p \mathrel{+}= \delta p, \quad v \mathrel{+}= \delta v, \quad q \leftarrow q \otimes \exp(\delta\theta), \quad b_a \mathrel{+}= \delta b_a, \quad b_g \mathrel{+}= \delta b_g$$

Correction	Measurement z	Predicted $h(x)$	Jacobian H (non-zero blocks)	Observes
Gravity (only if $\big\lvert\lvert a_m\rvert - g\big\rvert < 0.8$)	a_m	$b_a + R^\top(0,0,g)$	$H_{b_a} = I,\ H_\theta = [R^\top g_{up}]_\times$	roll, pitch, b_a
Leg velocity (each planted foot)	$-(\dot r_i + \omega \times r_i)$	$R^\top v$	$H_v = R^\top,\ H_\theta = [R^\top v]_\times$	v, attitude, b_g
Foot anchor (each planted foot)	touchdown point f_i	$p + R\,r_i$	$H_p = I,\ H_\theta = -R[r_i]_\times$	p, attitude, **yaw**

- **Foot anchor:** when a foot touches down, remember where $f_i = p + Rr_i$. While it stays planted any change in $p + Rr_i$ is estimation error. This stops position drift when standing and is the only thing that observes yaw. It is dropped when the foot lifts.
- **Flat floor:** the anchor's height is set to the known foot-centre height (0.033 m). Without this, each new anchor locked in the current height error and the estimate drifted up by 5–7 cm (this caused falls, §8).

- **Slip detection:** if the anchor innovation is statistically too large, $y^\top S^{-1} y > 16$ (a 3-DOF chi-square test, about 99.9 %), the foot is assumed to be slipping: it is re-anchored where it is and gives no correction. 5 cm of slip with the feet 0.3 m from the body centre would otherwise tilt the attitude estimate by about 10°.

> **WHAT CANNOT BE ESTIMATED**
>
> Roll, pitch, velocity and gyro bias are observable and accurate. **Yaw and absolute position** are only observable relative to the start (no compass, no GPS): they are initialised from the simulator and drift slowly while walking (about 1 m after 85 s). Walking does not need them because foot placement uses only relative positions. Accelerometer bias is only weakly observable (it looks like a small tilt) and stays inaccurate.

3.4 Contact (stance) information

While walking, the stance flags come from the gait schedule (§6), with the foot treated as planted only 0.05 s after touchdown so the landing impact is not used as a measurement. When standing without a schedule, a foot counts as planted if it is less than 3 cm above the lowest foot, using the complementary filter's attitude.

4. Body controller: virtual model control

> **IDEA**
>
> Instead of commanding joint angles, ask: *what net force and moment must the ground push on the body with?* Treat the body as a mass on virtual springs and dampers, add weight support, then work out which foot forces produce that wrench (§5) and which joint torques produce those forces. Because it works on forces, it automatically gives way when pushed and pushes back in proportion.

4.1 Standing: hold the pose

Desired pose: height $h^* = 0.50$ m, level, held position (x_0, y_0) and heading ψ_0 (set when standing starts).

$$F_{xy} = K_p \,\mathrm{clip}(p^*_{xy} - p_{xy}) - K_d\, v_{xy}, \qquad F_z = Mg + k_z(h^* - p_z) - d_z v_z$$

Gains: $K_p = 900$ N/m, $K_d = 220$ N·s/m (about 0.7 of critical damping $2\sqrt{K_p M}$); vertical $k_z = 1500$, $d_z = 300$. The position error is clipped to ±6 cm so a big shove does not demand more force than the feet can deliver.

4.2 Walking: track a commanded velocity

The commanded velocity (v_x, v_y) and yaw rate $\dot\psi^*$ come from `/cmd_vel`. With R_ψ the rotation by the *measured* heading, $v^* = R_\psi(v_x, v_y, 0)$:

$$F_{xy} = K_v\,(v^*_{xy} - v_{xy}) + \int K_i\,(v^*_{xy} - v_{xy})\,dt$$

$K_v = 160$ N·s/m, $K_i = 120$; the integral is limited to ±50 N and removes the steady speed shortfall caused by joint friction. Position is not held while walking.

4.3 Attitude and the friction limit

$$e_R = \tfrac{1}{2}\,\mathrm{vee}\big(R^{*\top}R - R^\top R^*\big), \qquad M_B = -K_R\, e_R - D_R\,(\omega - \omega^*), \qquad M = R\,M_B$$

R^* is level with heading ψ^*; e_R is zero when $R = R^*$ and otherwise a small rotation vector. Gains $K_R = (250, 300, 150)$ N·m/rad, $D_R = (14, 18, 10)$ N·m·s/rad. Higher damping (35–40) made the robot unstable: the body's roll inertia is well below the 1.5 kg·m² I first guessed, so damping tuned for a large inertia over-corrects.

Finally the horizontal force is limited so the feet do not slip:

$$|F_{xy}| \le 0.8\,\mu\,F_z, \qquad \mu = 0.6$$

The 0.8 is a safety margin below the friction cone. The resulting **wrench** $w = [F; M] \in \mathbb{R}^6$ goes to the force distribution.

4.4 Joint loop (inside the plugin, every step)

$$\tau_j = \text{clip}_{\pm 50}\Big(K_p\,(q_j^* - q_j) + K_d\,(\dot{q}_j^* - \dot{q}_j) + \tau_j^{ff}\Big)$$

Stance legs: $K_p = 90$, $K_d = 2$, $\dot{q}^* = 0$; q^* from the IK of the desired body pose over the feet's *current* world positions (so if a foot slips the target follows it), τ^{ff} from §5. Swing legs: stiffer, $K_p = 160$, $K_d = 4$, $\tau^{ff} = 0$, with the velocity feed-forward $\dot{q}^*$ taken from the swing trajectory. Before the estimator starts, a stiff Webots position servo holds the zero pose so the robot does not collapse while the filter waits for the robot to be still.

5. Force distribution and joint torques

5.1 Splitting the wrench over the feet

With foot forces f_i (ground pushing on foot i) at positions $\rho_i = R(r_i - c)$ relative to the centre of mass c, Newton–Euler balance is linear in the forces:

$$\underbrace{\begin{bmatrix} I & I & \cdots \\ [\rho_1]_\times & [\rho_2]_\times & \cdots \end{bmatrix}}_{A\ (6\times 3n)} \begin{bmatrix} f_1 \\ f_2 \\ \vdots \end{bmatrix} = \begin{bmatrix} F \\ M \end{bmatrix}$$

The first rows say the forces sum to the net force; the last say their moments about the CoM sum to the net moment. There are usually more unknowns than equations, so choose the smallest, most even forces that satisfy it within the friction limits:

$$\min_f \ \tfrac{1}{2}\|W(Af - w)\|^2 + \tfrac{\alpha}{2}\|f - f_{\text{ref}}\|^2 \quad \text{s.t.} \quad f_{\min} \le f_{z,i} \le f_{\max,i}, \ \ |f_{x,i}|, |f_{y,i}| \le \mu f_{z,i}$$

$f_{\text{ref}} = (F/n)$ spreads the load evenly; W scales newtons and newton-metres to comparable size. The closed-form unconstrained solution is the starting point, then projected gradient descent clamps each foot into its friction pyramid (a small 12-variable problem solved in under a millisecond in plain NumPy; no QP solver needed yet).

> **TWO FEET CANNOT DO EVERYTHING**
>
> In a trot only a diagonal pair is down for part of each cycle. Two contact points on a line can't produce a moment about that line, so A loses rank and roll about the diagonal is not controllable. The robot relies on short swing times (0.21 s) and on the next footfalls to correct it.

5.2 Forces to joint torques (virtual work)

A leg is nearly massless. For any small joint motion δq the work done by the motors, $\tau \cdot \delta q$, plus the work done by the ground force on the foot, $f \cdot J\delta q$, must be zero at equilibrium:

$$\tau \cdot \delta q + f \cdot J\,\delta q = 0 \quad \forall\, \delta q \quad \Longrightarrow \quad \boxed{\tau_i = -J_i^\top R^\top f_i}$$

($R^\top f_i$ expresses the world-frame force in the body frame, where J_i lives.) Swing legs get no force torque.

5.3 Load transfer

When a foot lands, full load on it at once caused pitch jolts and velocity spikes. Each stance foot's maximum vertical force is ramped: up over 0.10 s after touchdown and down over the last 0.05 s before lift-off, by scaling $f_{\max,i}$ in the constraint above; the other feet pick up the difference automatically.

6. Trot gait and foot placement

6.1 Contact schedule

Each leg has a phase $\varphi_i \in [0, 1)$ advancing at $1/T$, with period $T = 0.6$ s and duty factor $\beta = 0.65$:

$$\text{stance if } \varphi_i < \beta, \qquad \text{swing if } \varphi_i \geq \beta, \qquad s_i = \frac{\varphi_i - \beta}{1 - \beta} \in [0, 1] \text{ (swing progress)}$$

Phase offsets: FL and RR $= 0$, FR and RL $= 0.5$ (diagonal pairs move together). Because $\beta > 0.5$, both pairs are down for 30 % of the cycle, which gives a smooth hand-over:

Part of cycle	FL	FR	RL	RR	Feet down
0.00–0.15	stance	stance	stance	stance	4
0.15–0.50	stance	swing	swing	stance	2 (FL, RR)
0.50–0.65	stance	stance	stance	stance	4
0.65–1.00	swing	stance	stance	swing	2 (FR, RL)

Stopping: legs in stance freeze, legs in swing finish and land, then everything stands on four feet; starting again restarts from standing.

6.2 Swing trajectory

From lift-off point f_0 to landing goal f_g, with the horizontal motion shaped by a quintic so velocity and acceleration are zero at both ends (no jerk at lift-off or landing), and the foot lifted along an arch ($h = 6$ cm):

$$b(s) = 10s^3 - 15s^4 + 6s^5, \qquad f_{xy}(s) = f_{0,xy} + b(s)\,(f_{g,xy} - f_{0,xy}), \qquad f_z(s) = z_f + h \sin^2(\pi s)$$

The foot goal is converted to joint targets with the IK relative to the *actual* body pose, so the foot follows the world path regardless of body tilt; the target joint velocity is the derivative of that sequence.

6.3 Foot placement: the Raibert heuristic

Where to land? Roughly where the body will be by mid-stance, with a correction that makes the robot slow down if it is too fast and speed up if too slow:

$$p_i^{td} = p_{xy} + R_\psi n_i + \underbrace{\tfrac{1}{2} T_{st}\, \bar{v}}_{\text{move with the body}} + \underbrace{k_v\,(\bar{v} - v_{xy}^*)}_{\text{speed correction}} + \underbrace{\tfrac{1}{2} T_{st}\, \dot{\psi}^*\, [-n_y,\, n_x]^\top}_{\text{turning}}$$

n_i is the foot's nominal position under the body, $T_{st} = \beta T$ the stance time, $\bar{v}$ the estimated velocity low-pass filtered (time constant 0.15 s) so noise does not move the footsteps, $k_v = 0.2$ s (about $\sqrt{h/g}$, the "capture point" time constant of an inverted pendulum of height h). The landing goal is re-computed every step of the swing, so a push mid-swing changes where the foot lands.

> **WHY THE SPEED TERM MATTERS**
>
> For an inverted pendulum the point where the body would come to rest ("capture point") is $x + v\sqrt{h/g}$. Stepping there is exactly what a person does to avoid falling after a shove. The term $k_v(\bar{v} - v^*)$ is that rule.

6.4 Safety limits on touchdown

The step offset is limited to 10 cm; along the body axis the touchdown stays within ±12 cm of nominal; and a foot may never land more than 3 cm *inside* its nominal sideways position (the legs would cross under the body: this was a real failure, §8). The commanded heading is also kept within 0.2 rad of the measured one so the velocity target cannot run ahead of the robot.

7. How it resists pushes

A push changes the body velocity. The following layers react, from fastest to slowest:

Time scale	Mechanism	Equation
Immediately (8 ms)	Velocity damping on the stance feet pushes back; the attitude PD levels the body; the joint PDs resist deflection.	§4.1–4.4
Same step	Forces are clipped to the friction cone so the feet do not slide.	$\lvert F_{xy}\rvert \le 0.8\mu F_z$
Next footfall (within 0.2 s)	The swing foot lands farther in the push direction: capture-point stepping.	$k_v(\bar{v} - v^*)$ in §6.3
When standing and shoved hard	If speed > 0.25 m/s or tilt $> 9°$, the gait starts and takes recovery steps for at least 1.2 s, then stops and holds the new position.	recovery rule in the controller
Any time	The estimator stays valid: gravity correction is switched off while accelerating, slipping feet are detected and ignored.	§3.3

Measured while standing (force applied for 0.2 s at the body centre, Webots):

Push	Peak tilt	Peak displacement	Result
60 N sideways	0.7–1°	3–4 cm	recovers
100 N sideways / forward / backward	1.1–2.4°	6–10 cm	recovers
140 N sideways	2.2–3°	12–15 cm	recovers

> **HONEST STATUS**
> Repeated pushes without letting the robot settle sometimes still topple it (feet slide and the pose estimate drifts). Pushes *while walking* use the mechanisms above but have not yet been measured systematically; that test is next.

8. Numbers, lessons, next step

8.1 Key parameters

Robot / simulation	
Mass	27.2 kg
Thigh / calf	0.358 / 0.392 m
Stand height	0.50 m
Simulation step	8 ms
Torque limit	50 N·m / joint
Loop cost	about 3 ms per step

Gait	
Period / duty	0.6 s / 0.65
Swing height	6 cm
k_v	0.2 s
Speed limits	0.5 m/s fwd, 0.25 sideways
Yaw rate limit	0.8 rad/s
Walked (no push)	about 4 min on a 1 m circle

Estimator result (Webots, scored against GPS + inertial unit)	Value
Complementary filter roll/pitch error	0.1–0.6°
EKF roll/pitch / yaw error (standing)	0.1–0.8° / about 0.2°
Velocity error / position error (standing)	0.01–0.03 m/s / about 1 cm
Estimated gyro bias vs true	(0.010, −0.008, 0.006) vs (0.010, −0.008, 0.005) rad/s

8.2 Lessons from debugging (each one a real failure)

Symptom	Cause	Fix
Robot collapsed when torque control started	Controller ran as a ROS node: 10s of ms of delay	Run estimator and controller inside the simulation step
Oscillated and fell after a large push	Position damping only 0.3 of critical; force beyond friction	Damping near critical; friction-cone clamp
Fell after higher attitude damping	Over-damping with a smaller real inertia	Keep $D_R \approx 14$–18
Position estimate corrupted after slips	Anchor update treated slip as body motion	Chi-square slip gate and re-anchor
Walked 30 s, then suddenly fell	Height estimate drifted +5–7 cm; heading target ran ahead in turns; abrupt load transfer; legs crossed under the body	Flat-floor anchor height; heading lead limit; force ramps; touchdown limits

8.3 Next: Model Predictive Control (not built yet)

The current controller reacts to the present state. MPC would *plan* the ground forces over the next second using the gait's contact schedule. A standard single-rigid-body model with the state $x = [\Theta,\, p,\, \omega,\, v]$ (attitude, position, angular and linear velocity):

$$\dot{p} = v, \quad \dot{v} = \frac{1}{M}\sum_i f_i + g_W, \quad \dot{\omega} \approx I^{-1}\Big(\sum_i \rho_i \times f_i\Big), \quad \dot{\Theta} \approx E(\psi)\,\omega$$

$$\min_{f_{0:N-1}} \sum_{k=0}^{N-1} \left\|x_k - x_k^{\text{ref}}\right\|_Q^2 + \|f_k\|_R^2 \quad \text{s.t. } x_{k+1} = A_k x_k + B_k f_k + g, \;\; f_k \text{ in the friction pyramid}, \;\; f_{k,i} = 0 \text{ for feet in swing}$$

This is a quadratic program solved every 20–30 ms; the first force is sent to the same torque mapping $\tau = -J^\top R^\top f$ used now, and the state comes from the EKF in §3. It needs a QP solver (such as OSQP), which is not installed here yet. The pieces already built (estimator, force-to-torque mapping, contact schedule, swing legs) are exactly its inputs and outputs, so MPC replaces §4.1–4.3 and the force-distribution step rather than starting over.

Code: `spot_controller/` (kinematics: `ik_solver.py`, `spot_4leg_ik.py`; estimation: `attitude_filter.py`, `ekf.py`, `leg_odometry.py`, `estimator_core.py`; control: `locomotion_core.py`, `force_distribution.py`, `gait.py`; Webots bridge: `webots_plugin.py`). 30 automated tests cover the maths offline.